EECT 6325 – VLSI Design
Project 2: CAN Bus Controller bitstream

Corey Finch (cxf210015)
Adil Bhojwani (axb210168)
Paresh Grover (pxg210048)
Date: 9/23/2026

I. General Description

This design implements a scalable CAN 2.0B controller in Verilog that supports both transmission and reception of CAN frames through a configurable register-based interface. The controller manages multiple transmit mailboxes, configurable acceptance filtering, a receive queue, CAN bit timing, CRC generation, bit stuffing, arbitration, acknowledgment, and error handling. A finite state machine sequences the CAN protocol through initialization, idle, arbitration, control, data, CRC, acknowledgment, end-of-frame, interframe, error, and bus-off states. Transmit frames are selected from the available mailboxes based on CAN arbitration priority, while received frames are decoded and stored in the receive queue when they satisfy the configured acceptance filters. The design includes a dedicated bit-timing module for synchronization and sampling, as well as a CRC-15 module for CAN error detection. Error conditions such as bit, stuffing, CRC, form, and acknowledgment errors are detected and used to update the transmit and receive error counters and controller status. The controller provides transmit and receive status information, interrupt flags, error codes, bus-off detection, and a loopback mode through its register interface. The design is parameterized to allow the number of transmit mailboxes, acceptance filters, and receive queue depth to be adjusted without changing the overall controller architecture. For Project 2, the behavioral Verilog is synthesized using Synopsys Design Vision and mapped to the provided standard-cell library to generate a structural netlist and determine the total cell count. The mapped Verilog is then simulated and its waveforms are compared with the original behavioral implementation to verify that both designs maintain the same functionality.

II. Design Vision Cell Report Snippet

This snippet shows several D flip-flop (DFF) cells used for storing the controller state and other registered signals and the final result showing a total of 3,594 cells.

```
state_reg[3]            dff             library         7.000000  n
state_reg[4]            dff             library         7.000000  n
tec_reg[0]              dff             library         7.000000  n
tec_reg[1]              dff             library         7.000000  n
tec_reg[2]              dff             library         7.000000  n
tec_reg[3]              dff             library         7.000000  n
tec_reg[4]              dff             library         7.000000  n
tec_reg[5]              dff             library         7.000000  n
tec_reg[6]              dff             library         7.000000  n
tec_reg[7]              dff             library         7.000000  n
tec_reg[8]              dff             library         7.000000  n
transmitting_reg        dff             library         7.000000  n
tx_int_reg              dff             library         7.000000  n
tx_sof_reg              dff             library         7.000000  n
was_tx_reg              dff             library         7.000000  n
--------------------------------------------------------------------------------
Total 3594 cells                                        8580.000000
```

Figure 1: Snippet of the cell report showing DFF references sourced from “library” and a 3,594 cell count.

III. Behavioral and Mapped VCD Waveforms

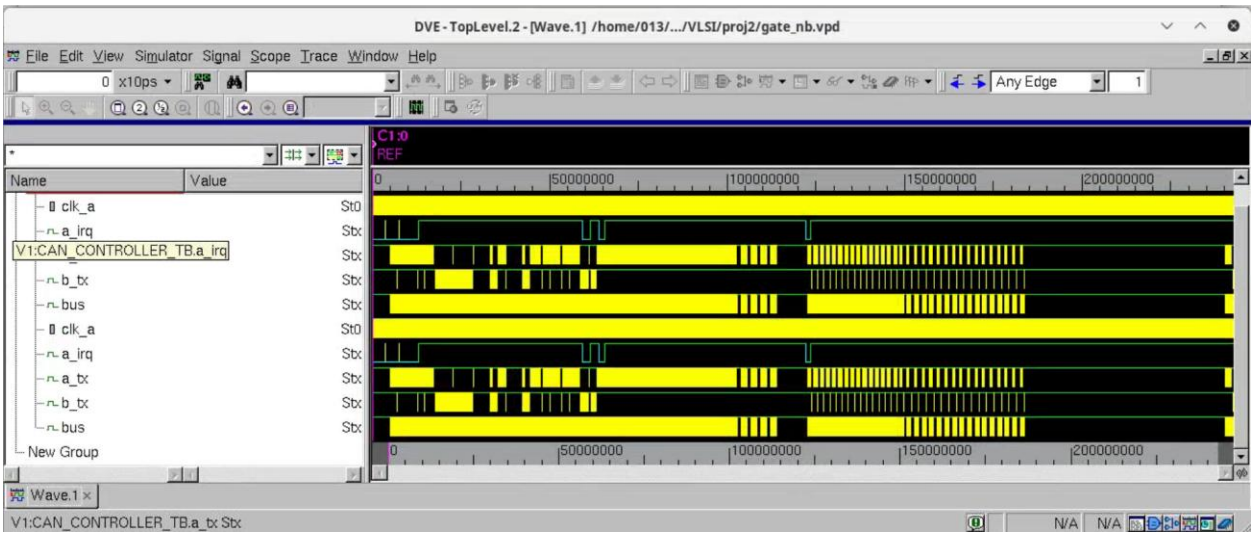


Figure 2: Behavioral simulation (top five signals) and mapped netlist simulation (bottom five signals) for the full 2.43 ms test run. Time axis in units of 10 ps.

The same self-checking testbench was run on the behavioral Verilog and on the mapped gate-level netlist produced by Design Vision. The testbench places three CAN controllers on a

shared bus. Node A is the controller under test, node B runs on a clock 1% slower to exercise bit resynchronization, and node C acts as a third bus node. Figure X shows node A's clock (clk_a), node A's interrupt output (a_irq), the transmit pins of nodes A and B (a_tx, b_tx), and the shared bus (bus) for both simulations.

The two sets of waveforms are identical across the entire run. The major phases of the test are visible in both:

- **Normal traffic (left):** frame exchanges between nodes, including arbitration where A and B transmit simultaneously and the lower identifier wins.
- **ACK errors (middle):** a long block of activity on `a_tx` and `bus` while `b_tx` stays idle. Node A is alone on the bus and repeatedly retransmits a frame that no node acknowledges, until it becomes error passive.
- **Bus-off (right of center):** a dense section on both `a_tx` and `b_tx`. Deliberately corrupted frames cause error frames from both nodes until node A's error counter drives it into bus-off.
- **Recovery (far right):** a quiet stretch while node A waits the required 128 × 11 recessive bits, followed by the final burst, where node A successfully retransmits its pending frame.

Both simulations pass every check in the testbench and end at the exact same simulation time, 242,886,820 × 10 ps. Every bus bit, retransmission and register access therefore occurred on the same clock cycle in both designs, which shows the mapped netlist is functionally identical to the behavioral code.

A note on the gate-level modules. When the mapped netlist was simulated in VCS using the provided header.v, every check passed except one: a status flag that should have been set to 1 read as 0. The cause is how header.v models flip-flops, not the design. In real hardware, all flip-flops capture their inputs at the same instant on each clock edge. The provided model instead lets the simulator update them one at a time. Depending on the order it picks, one flip-flop can see another's value from the next cycle too early. Here, that allowed the flag to miss a signal lasting only one clock cycle. After the flip-flop model was corrected so they all update together and no further modifications, the same netlist passed every check and matched the behavioral simulation exactly. The waveforms in Figure 2 use this corrected model. The netlist also passes every check with the original header.v in a second simulator, Icarus Verilog, confirming the issue comes from the simulator's update order rather than the design.

IV. Scaled-up Code

```
module CAN_CONTROLLER #(
    parameter NUM_TX_MBOX = 2,
    parameter NUM_FILTERS = 1,
    parameter RX_DEPTH    = 1
)(
    input            clk,
    input            reset,
    input      [7:0] addr,
    input      [7:0] wdata,
    input            we,
    output reg [7:0] rdata,
    output           irq,
    input            rx,
    output           tx
);

    localparam ST_OFF     = 5'd0,  ST_INTEGRATE = 5'd1,  ST_IDLE    = 5'd2,
               ST_ID_A    = 5'd3,  ST_SRR       = 5'd4,  ST_IDE     = 5'd5,
               ST_ID_B    = 5'd6,  ST_RTR       = 5'd7,  ST_R1      = 5'd8,
               ST_R0      = 5'd9,  ST_DLC       = 5'd10, ST_DATA    = 5'd11,
               ST_CRC     = 5'd12, ST_CRC_DEL   = 5'd13, ST_ACK     = 5'd14,
               ST_ACK_DEL = 5'd15, ST_EOF       = 5'd16, ST_INTER   = 5'd17,
               ST_SUSPEND = 5'd18, ST_ERR_FLAG  = 5'd19, ST_ERR_DEL = 5'd20,
               ST_BUSOFF  = 5'd21;

    localparam E_NONE = 3'd0, E_BIT = 3'd1, E_STUFF = 3'd2,
               E_CRC  = 3'd3, E_FORM = 3'd4, E_ACK  = 3'd5;

    integer i_sel, i_flt, i_wm, i_wf, i_wk, i_q, i_rf, i_rm, i_rst;

    reg        en, loopback;
    reg  [7:0] int_en, int_flag;
    reg  [5:0] brp;
    reg  [1:0] sjw_m1;
    reg  [2:0] prop_m1, ph1_m1, ph2_m1;
    reg  [2:0] err_code;

    reg        mb_req  [0:NUM_TX_MBOX-1];
    reg        mb_lost [0:NUM_TX_MBOX-1];
    reg        mb_err  [0:NUM_TX_MBOX-1];
    reg [28:0] mb_id   [0:NUM_TX_MBOX-1];
    reg        mb_ide  [0:NUM_TX_MBOX-1];
    reg        mb_rtr  [0:NUM_TX_MBOX-1];
    reg  [3:0] mb_dlc  [0:NUM_TX_MBOX-1];
    reg [63:0] mb_data [0:NUM_TX_MBOX-1];

    reg        f_en   [0:NUM_FILTERS-1];
    reg        f_ide  [0:NUM_FILTERS-1];
    reg [28:0] f_id   [0:NUM_FILTERS-1];
    reg [28:0] f_mask [0:NUM_FILTERS-1];

    reg [28:0] q_id   [0:RX_DEPTH-1];
    reg        q_ide  [0:RX_DEPTH-1];
    reg        q_rtr  [0:RX_DEPTH-1];
    reg  [3:0] q_dlc  [0:RX_DEPTH-1];
    reg [63:0] q_data [0:RX_DEPTH-1];
    reg  [3:0] q_hit  [0:RX_DEPTH-1];
    reg  [2:0] q_wr, q_rd, q_count;
    reg        q_ovf;

    reg  [4:0] state;
    reg  [6:0] bcnt;
    reg        tx_int;
    reg        tx_sof;
    reg        transmitting;
    reg        was_tx;
    reg  [1:0] cur_mb;
    reg        s_on, s_last;
    reg  [2:0] s_cnt;
    reg [28:0] r_id;
    reg        r_srr, r_ide, r_rtr;
    reg  [3:0] r_dlc;
    reg [63:0] r_data;
    reg  [6:0] align_cnt;
    reg  [8:0] tec;
```

```
    reg  [7:0] rec;
    reg  [6:0] boff_cnt;
    reg        rx_meta, rx_s;
    reg        ev_tx_ok, ev_tx_err, ev_arb_lost, ev_rx_frame, ev_err, ev_busoff;
    reg  [2:0] ev_code;

    wire passive     = (tec > 9'd127) || (rec > 8'd127);
    wire warning     = (tec >= 9'd96) || (rec >= 8'd96);
    wire in_arb      = (state >= ST_ID_A) && (state <= ST_RTR);
    wire in_stuffed  = (state >= ST_ID_A) && (state <= ST_CRC);
    wire in_frame    = (state >= ST_ID_A) && (state <= ST_EOF);
    wire bus_idle_st = (state == ST_IDLE) || (state == ST_INTER) || (state == ST_SUSPEND);
    wire stuff_next  = s_on && (s_cnt == 3'd5);
    wire [6:0] r_bits = (r_dlc > 4'd8) ? 7'd64 : {r_dlc, 3'b000};

    wire tx_point, sample_point, sampled;
    wire hard_sync_en = (state == ST_INTEGRATE) || bus_idle_st || (state == ST_BUSOFF);

    CAN_BTL btl (
        .clk(clk), .reset(reset), .enable(en),
        .brp(brp), .sjw_m1(sjw_m1), .prop_m1(prop_m1), .ph1_m1(ph1_m1), .ph2_m1(ph2_m1),
        .rx_s(rx_s), .hard_sync_en(hard_sync_en),
        .resync_en(!hard_sync_en && state != ST_OFF),
        .tx_point(tx_point), .sample_point(sample_point), .sampled(sampled)
    );

    wire [14:0] crc;
    CAN_CRC15 crc_gen (
        .clk(clk), .reset(reset),
        .clear(sample_point && bus_idle_st && !sampled),
        .enable(sample_point && in_stuffed && !stuff_next),
        .bit_in(sampled), .crc(crc)
    );

    wire [28:0] t_id   = mb_id[cur_mb];
    wire        t_ide  = mb_ide[cur_mb];
    wire        t_rtr  = mb_rtr[cur_mb];
    wire  [3:0] t_dlc  = mb_dlc[cur_mb];
    wire [63:0] t_data = mb_data[cur_mb];
    wire        tx_busy = transmitting || tx_sof;

    reg frame_bit;
    always @(*) begin
        case (state)
            ST_ID_A: frame_bit = t_id[28 - bcnt];
            ST_SRR:  frame_bit = t_ide | t_rtr;
            ST_IDE:  frame_bit = t_ide;
            ST_ID_B: frame_bit = t_id[17 - bcnt];
            ST_RTR:  frame_bit = t_rtr;
            ST_DLC:  frame_bit = t_dlc[3 - bcnt];
            ST_DATA: frame_bit = t_data[63 - bcnt];
            ST_CRC:  frame_bit = crc[14];
            default: frame_bit = 1'b0;
        endcase
    end

    reg        sel_valid;
    reg  [1:0] sel_idx;
    reg [31:0] sel_key, key;
    always @(*) begin
        sel_valid = 1'b0;
        sel_idx   = 2'd0;
        sel_key   = 32'hFFFFFFFF;
        for (i_sel = 0; i_sel < NUM_TX_MBOX; i_sel = i_sel + 1) begin

            key = {mb_id[i_sel][28:18], mb_ide[i_sel] | mb_rtr[i_sel], mb_ide[i_sel],
                   mb_id[i_sel][17:0] & {18{mb_ide[i_sel]}}, mb_rtr[i_sel] & mb_ide[i_sel]};
            if (mb_req[i_sel] && (!sel_valid || key < sel_key)) begin
                sel_valid = 1'b1;
                sel_idx   = i_sel;
                sel_key   = key;
            end
        end
    end

    reg        any_en, hit;
    reg  [2:0] hit_idx;
    reg [28:0] fmask;
```

```
    always @(*) begin
        any_en  = 1'b0;
        hit     = 1'b0;
        hit_idx = 3'd0;
        fmask   = 29'd0;
        for (i_flt = NUM_FILTERS-1; i_flt >= 0; i_flt = i_flt - 1) begin
            fmask = {f_mask[i_flt][28:18], f_mask[i_flt][17:0] & {18{f_ide[i_flt]}}};
            if (f_en[i_flt]) begin
                any_en = 1'b1;
                if (f_ide[i_flt] == r_ide && ((r_id ^ f_id[i_flt]) & fmask) == 29'd0) begin
                    hit     = 1'b1;
                    hit_idx = i_flt;
                end
            end
        end
    end
    wire rx_accept = !any_en || hit;
    wire rx_push   = ev_rx_frame && rx_accept && (q_count != RX_DEPTH);
    wire rx_ovf    = ev_rx_frame && rx_accept && (q_count == RX_DEPTH);
    wire rx_pop    = we && (addr == 8'h50) && wdata[0] && (q_count != 3'd0);

    reg [2:0] err_type;
    always @(*) begin
        err_type = E_NONE;
        if (stuff_next) begin
            if (sampled == s_last) err_type = E_STUFF;
        end
        else if (!sampled && (state == ST_CRC_DEL || state == ST_ACK_DEL ||
                 (state == ST_EOF && (bcnt != 7'd6 || transmitting))))
            err_type = E_FORM;
        else if (transmitting && state == ST_ACK && sampled && !loopback)
            err_type = E_ACK;
        else if (transmitting && state != ST_ACK && tx_int != sampled && !(in_arb && tx_int))
            err_type = E_BIT;
        else if (!transmitting && state == ST_ACK_DEL && crc != 15'd0)
            err_type = E_CRC;
    end
    wire arb_lost = transmitting && in_arb && tx_int && !sampled && !stuff_next;

    assign tx = loopback ? 1'b1 : tx_int;

    always @(posedge clk) begin
        if (reset) begin
            rx_meta <= 1'b1;
            rx_s    <= 1'b1;
        end
        else begin
            rx_meta <= loopback ? tx_int : rx;
            rx_s    <= rx_meta;
        end
    end

    always @(posedge clk) begin
        ev_tx_ok <= 1'b0;  ev_tx_err <= 1'b0;  ev_arb_lost <= 1'b0;
        ev_rx_frame <= 1'b0;  ev_err <= 1'b0;  ev_busoff <= 1'b0;

        if (reset || !en) begin
            state <= ST_OFF;  bcnt <= 7'd0;  tx_int <= 1'b1;  tx_sof <= 1'b0;
            transmitting <= 1'b0;  was_tx <= 1'b0;  cur_mb <= 2'd0;
            s_on <= 1'b0;  s_last <= 1'b1;  s_cnt <= 3'd0;
            tec <= 9'd0;  rec <= 8'd0;  boff_cnt <= 7'd0;  ev_code <= E_NONE;
            align_cnt <= 7'd0;
        end
        else begin

            if (align_cnt != 7'd0) begin
                r_data    <= {r_data[62:0], 1'b0};
                align_cnt <= align_cnt - 7'd1;
            end

            if (state == ST_OFF) begin
                state <= ST_INTEGRATE;
                bcnt  <= 7'd0;
            end

            if (tx_point) begin
                tx_sof <= 1'b0;
                if (state == ST_IDLE && sel_valid) begin
```

```
                tx_int <= 1'b0;
                tx_sof <= 1'b1;
                cur_mb <= sel_idx;
            end
            else if (transmitting && stuff_next)
                tx_int <= ~s_last;
            else if (transmitting && in_stuffed)
                tx_int <= frame_bit;
            else if (state == ST_ACK && !transmitting && crc == 15'd0)
                tx_int <= 1'b0;
            else if (state == ST_ERR_FLAG && !passive)
                tx_int <= 1'b0;
            else
                tx_int <= 1'b1;
        end

        if (sample_point) begin
            case (state)
                ST_OFF: ;
                ST_INTEGRATE: begin
                    if (!sampled)            bcnt  <= 7'd0;
                    else if (bcnt == 7'd10) begin state <= ST_IDLE; bcnt <= 7'd0; end
                    else                     bcnt  <= bcnt + 7'd1;
                end
                ST_BUSOFF: begin
                    if (!sampled) bcnt <= 7'd0;
                    else if (bcnt == 7'd10) begin
                        bcnt <= 7'd0;
                        if (boff_cnt == 7'd127) begin
                            boff_cnt <= 7'd0;  tec <= 9'd0;  rec <= 8'd0;
                            state    <= ST_IDLE;
                        end
                        else boff_cnt <= boff_cnt + 7'd1;
                    end
                    else bcnt <= bcnt + 7'd1;
                end
                ST_IDLE, ST_INTER, ST_SUSPEND: begin
                    if (!sampled) begin
                        state <= ST_ID_A;  bcnt <= 7'd0;
                        transmitting <= tx_sof;
                        s_on  <= 1'b1;  s_last <= 1'b0;  s_cnt <= 3'd1;
                        r_id  <= 29'd0;  r_dlc <= 4'd0;
                        r_ide <= 1'b0;   r_rtr  <= 1'b0;   r_srr <= 1'b0;
                    end
                    else if (state == ST_INTER) begin
                        if (bcnt == 7'd2) begin
                            state <= (was_tx && passive) ? ST_SUSPEND : ST_IDLE;
                            bcnt  <= 7'd0;
                        end
                        else bcnt <= bcnt + 7'd1;
                    end
                    else if (state == ST_SUSPEND) begin
                        if (bcnt == 7'd7) begin state <= ST_IDLE; bcnt <= 7'd0; end
                        else bcnt <= bcnt + 7'd1;
                    end
                end
                ST_ERR_FLAG: begin
                    if (bcnt == 7'd5) begin state <= ST_ERR_DEL; bcnt <= 7'd0; end
                    else bcnt <= bcnt + 7'd1;
                end
                ST_ERR_DEL: begin
                    if (!sampled)           bcnt <= 7'd0;
                    else if (bcnt == 7'd7) begin state <= ST_INTER; bcnt <= 7'd0; end
                    else                    bcnt <= bcnt + 7'd1;
                end
                default: begin
                    if (err_type != E_NONE) begin
                        ev_err  <= 1'b1;
                        ev_code <= err_type;
                        s_on    <= 1'b0;
                        bcnt    <= 7'd0;
                        was_tx  <= transmitting;
                        transmitting <= 1'b0;
                        state   <= ST_ERR_FLAG;
                        if (transmitting) begin
                            ev_tx_err <= 1'b1;

                            if (!(err_type == E_ACK && passive)) begin
```

```
                tec <= tec + 9'd8;
                if (tec >= 9'd248) begin
                    state     <= ST_BUSOFF;
                    boff_cnt  <= 7'd0;
                    ev_busoff <= 1'b1;
                end
            end
        end
        else if (rec != 8'hFF) rec <= rec + 8'd1;
    end
    else if (stuff_next) begin
        s_last <= sampled;
        s_cnt  <= 3'd1;
    end
    else begin
        if (arb_lost) begin
            transmitting <= 1'b0;
            ev_arb_lost  <= 1'b1;
        end
        if (in_stuffed) begin
            if (sampled == s_last) s_cnt <= s_cnt + 3'd1;
            else begin s_last <= sampled; s_cnt <= 3'd1; end
        end
        else s_on <= 1'b0;

        case (state)
            ST_ID_A: begin
                r_id <= {r_id[27:0], sampled};
                if (bcnt == 7'd10) begin state <= ST_SRR; bcnt <= 7'd0; end
                else bcnt <= bcnt + 7'd1;
            end
            ST_SRR: begin r_srr <= sampled; state <= ST_IDE; end
            ST_IDE: begin
                r_ide <= sampled;
                if (sampled) begin state <= ST_ID_B; bcnt <= 7'd0; end
                else begin
                    r_rtr <= r_srr;
                    r_id  <= {r_id[10:0], 18'd0};
                    state <= ST_R0;
                end
            end
            ST_ID_B: begin
                r_id <= {r_id[27:0], sampled};
                if (bcnt == 7'd17) begin state <= ST_RTR; bcnt <= 7'd0; end
                else bcnt <= bcnt + 7'd1;
            end
            ST_RTR:  begin r_rtr <= sampled; state <= ST_R1; end
            ST_R1:   state <= ST_R0;
            ST_R0:   begin state <= ST_DLC; bcnt <= 7'd0; end
            ST_DLC: begin
                r_dlc <= {r_dlc[2:0], sampled};
                if (bcnt == 7'd3) begin
                    bcnt <= 7'd0;
                    if (r_rtr || {r_dlc[2:0], sampled} == 4'd0) begin
                        state     <= ST_CRC;
                        align_cnt <= 7'd64;
                    end
                    else state <= ST_DATA;
                end
                else bcnt <= bcnt + 7'd1;
            end
            ST_DATA: begin
                r_data <= {r_data[62:0], sampled};
                if (bcnt == r_bits - 7'd1) begin
                    state     <= ST_CRC;
                    bcnt      <= 7'd0;
                    align_cnt <= 7'd64 - r_bits;
                end
                else bcnt <= bcnt + 7'd1;
            end
            ST_CRC: begin
                if (bcnt == 7'd14) begin state <= ST_CRC_DEL; bcnt <= 7'd0; end
                else bcnt <= bcnt + 7'd1;
            end
            ST_CRC_DEL: state <= ST_ACK;
            ST_ACK:     state <= ST_ACK_DEL;
            ST_ACK_DEL: begin state <= ST_EOF; bcnt <= 7'd0; end
            ST_EOF: begin
```

```
                                if (bcnt == 7'd5 && (!transmitting || loopback)) begin
                                    ev_rx_frame <= 1'b1;
                                    if (rec > 8'd127)     rec <= 8'd120;
                                    else if (rec != 8'd0) rec <= rec - 8'd1;
                                end
                                if (bcnt == 7'd6) begin
                                    if (transmitting) begin
                                        ev_tx_ok <= 1'b1;
                                        if (tec != 9'd0) tec <= tec - 9'd1;
                                    end
                                    was_tx <= transmitting;
                                    transmitting <= 1'b0;
                                    state <= ST_INTER;
                                    bcnt  <= 7'd0;
                                end
                                else bcnt <= bcnt + 7'd1;
                            end
                            default: state <= ST_IDLE;
                        endcase
                    end
                end
            endcase
        end
    end
end

wire [7:0] int_set = {ev_busoff, ev_err, rx_ovf, rx_push,
                      ev_tx_ok ? (4'b0001 << cur_mb) : 4'b0000};

always @(posedge clk) begin
    if (reset) begin
        en <= 1'b0;  loopback <= 1'b0;  int_en <= 8'd0;  int_flag <= 8'd0;
        brp <= 6'd3;  sjw_m1 <= 2'd1;
        prop_m1 <= 3'd1;  ph1_m1 <= 3'd2;  ph2_m1 <= 3'd3;
        err_code <= E_NONE;
        for (i_rst = 0; i_rst < NUM_TX_MBOX; i_rst = i_rst + 1) begin
            mb_req[i_rst] <= 1'b0;  mb_lost[i_rst] <= 1'b0;  mb_err[i_rst] <= 1'b0;
        end
        for (i_wf = 0; i_wf < NUM_FILTERS; i_wf = i_wf + 1) f_en[i_wf] <= 1'b0;
    end
    else begin
        if (we && addr == 8'h00) begin en <= wdata[0]; loopback <= wdata[1]; end
        if (we && addr == 8'h02) int_en <= wdata;
        if (we && addr == 8'h04) begin brp <= wdata[5:0]; sjw_m1 <= wdata[7:6]; end
        if (we && addr == 8'h05) begin prop_m1 <= wdata[2:0]; ph1_m1 <= wdata[5:3]; end
        if (we && addr == 8'h06) ph2_m1 <= wdata[2:0];
        int_flag <= ((we && addr == 8'h03) ? (int_flag & ~wdata) : int_flag) | int_set;
        if (ev_err) err_code <= ev_code;

        for (i_wm = 0; i_wm < NUM_TX_MBOX; i_wm = i_wm + 1) begin

            if (we && addr[7:4] == i_wm + 1 &&
                (addr[3:0] == 4'd0 || !(mb_req[i_wm] || (tx_busy && cur_mb == i_wm)))) begin
                case (addr[3:0])
                    4'd0: begin
                        mb_req[i_wm]  <= wdata[0];
                        mb_lost[i_wm] <= 1'b0;
                        mb_err[i_wm]  <= 1'b0;
                    end
                    4'd1: mb_id[i_wm][28:24] <= wdata[4:0];
                    4'd2: mb_id[i_wm][23:16] <= wdata;
                    4'd3: mb_id[i_wm][15:8]  <= wdata;
                    4'd4: mb_id[i_wm][7:0]   <= wdata;
                    4'd5: begin
                        mb_ide[i_wm] <= wdata[7];
                        mb_rtr[i_wm] <= wdata[6];
                        mb_dlc[i_wm] <= wdata[3:0];
                    end
                    default: ;
                endcase
                for (i_wk = 0; i_wk < 8; i_wk = i_wk + 1)
                    if (addr[3:0] == i_wk + 6) mb_data[i_wm][63-8*i_wk -: 8] <= wdata;
            end
            if (we && addr == 8'h00 && wdata[2]) mb_req[i_wm] <= 1'b0;
            if (cur_mb == i_wm) begin
                if (ev_tx_ok)    mb_req[i_wm]  <= 1'b0;
                if (ev_arb_lost) mb_lost[i_wm] <= 1'b1;
```

```
                if (ev_tx_err)   mb_err[i_wm]  <= 1'b1;
            end
        end

        for (i_wf = 0; i_wf < NUM_FILTERS; i_wf = i_wf + 1) begin
            if (we && addr[7] && addr[6:4] == i_wf) begin
                case (addr[3:0])
                    4'd0: begin f_en[i_wf] <= wdata[0]; f_ide[i_wf] <= wdata[1]; end
                    4'd1: f_id[i_wf][28:24]   <= wdata[4:0];
                    4'd2: f_id[i_wf][23:16]   <= wdata;
                    4'd3: f_id[i_wf][15:8]    <= wdata;
                    4'd4: f_id[i_wf][7:0]     <= wdata;
                    4'd5: f_mask[i_wf][28:24] <= wdata[4:0];
                    4'd6: f_mask[i_wf][23:16] <= wdata;
                    4'd7: f_mask[i_wf][15:8]  <= wdata;
                    4'd8: f_mask[i_wf][7:0]   <= wdata;
                    default: ;
                endcase
            end
        end
    end
end

always @(posedge clk) begin
    if (reset) begin
        q_wr <= 3'd0;  q_rd <= 3'd0;  q_count <= 3'd0;  q_ovf <= 1'b0;
    end
    else begin
        if (rx_push) begin
            for (i_q = 0; i_q < RX_DEPTH; i_q = i_q + 1)
                if (q_wr == i_q) begin
                    q_id[i_q]  <= r_id;   q_ide[i_q]  <= r_ide;   q_rtr[i_q] <= r_rtr;
                    q_dlc[i_q] <= r_dlc;  q_data[i_q] <= r_data;  q_hit[i_q] <= {hit, hit_idx};
                end
            q_wr <= (q_wr == RX_DEPTH-1) ? 3'd0 : q_wr + 3'd1;
        end
        if (rx_pop) q_rd <= (q_rd == RX_DEPTH-1) ? 3'd0 : q_rd + 3'd1;
        q_count <= q_count + {2'b00, rx_push} - {2'b00, rx_pop};
        if (we && addr == 8'h50 && wdata[1]) q_ovf <= 1'b0;
        if (rx_ovf) q_ovf <= 1'b1;
    end
end

function [7:0] frame_byte;
    input [3:0]  off;
    input [28:0] id;
    input        ide, rtr;
    input [3:0]  dlc;
    input [63:0] data;
    begin
        case (off)
            4'd1:  frame_byte = {3'b000, id[28:24]};
            4'd2:  frame_byte = id[23:16];
            4'd3:  frame_byte = id[15:8];
            4'd4:  frame_byte = id[7:0];
            4'd5:  frame_byte = {ide, rtr, 2'b00, dlc};
            4'd6:  frame_byte = data[63:56];
            4'd7:  frame_byte = data[55:48];
            4'd8:  frame_byte = data[47:40];
            4'd9:  frame_byte = data[39:32];
            4'd10: frame_byte = data[31:24];
            4'd11: frame_byte = data[23:16];
            4'd12: frame_byte = data[15:8];
            4'd13: frame_byte = data[7:0];
            default: frame_byte = 8'h00;
        endcase
    end
endfunction

always @(*) begin
    rdata = 8'h00;
    if (addr[7:4] == 4'h0) begin
        case (addr[3:0])
            4'd0: rdata = {6'd0, loopback, en};
            4'd1: rdata = {state == ST_IDLE, q_count == RX_DEPTH, q_count != 3'd0,
                           state == ST_BUSOFF, passive, warning,
                           in_frame && !transmitting, transmitting};
            4'd2: rdata = int_en;
```

```
                4'd3: rdata = int_flag;
                4'd4: rdata = {sjw_m1, brp};
                4'd5: rdata = {2'b00, ph1_m1, prop_m1};
                4'd6: rdata = {5'd0, ph2_m1};
                4'd7: rdata = tec[8] ? 8'hFF : tec[7:0];
                4'd8: rdata = rec;
                4'd9: rdata = {5'd0, err_code};
                default: rdata = 8'h00;
            endcase
        end
        else if (addr[7:4] == 4'h5) begin
            if (addr[3:0] == 4'd0)       rdata = {3'b000, q_count, q_ovf, q_count != 3'd0};
            else if (addr[3:0] == 4'd14) rdata = {4'd0, q_hit[q_rd]};
            else rdata = frame_byte(addr[3:0], q_id[q_rd], q_ide[q_rd], q_rtr[q_rd],
                                    q_dlc[q_rd], q_data[q_rd]);
        end
        else if (addr[7]) begin
            for (i_rf = 0; i_rf < NUM_FILTERS; i_rf = i_rf + 1)
                if (addr[6:4] == i_rf) begin
                    case (addr[3:0])
                        4'd0: rdata = {6'd0, f_ide[i_rf], f_en[i_rf]};
                        4'd1: rdata = {3'b000, f_id[i_rf][28:24]};
                        4'd2: rdata = f_id[i_rf][23:16];
                        4'd3: rdata = f_id[i_rf][15:8];
                        4'd4: rdata = f_id[i_rf][7:0];
                        4'd5: rdata = {3'b000, f_mask[i_rf][28:24]};
                        4'd6: rdata = f_mask[i_rf][23:16];
                        4'd7: rdata = f_mask[i_rf][15:8];
                        4'd8: rdata = f_mask[i_rf][7:0];
                        default: rdata = 8'h00;
                    endcase
                end
        end
        else begin
            for (i_rm = 0; i_rm < NUM_TX_MBOX; i_rm = i_rm + 1)
                if (addr[7:4] == i_rm + 1)
                    rdata = (addr[3:0] == 4'd0)
                          ? {5'd0, mb_err[i_rm], mb_lost[i_rm], mb_req[i_rm]}
                          : frame_byte(addr[3:0], mb_id[i_rm], mb_ide[i_rm], mb_rtr[i_rm],
                                       mb_dlc[i_rm], mb_data[i_rm]);
        end
    end

    assign irq = |(int_flag & int_en);

endmodule

module CAN_BTL (
    input            clk,
    input            reset,
    input            enable,
    input      [5:0] brp,
    input      [1:0] sjw_m1,
    input      [2:0] prop_m1,
    input      [2:0] ph1_m1,
    input      [2:0] ph2_m1,
    input            rx_s,
    input            hard_sync_en,
    input            resync_en,
    output reg       tx_point,
    output reg       sample_point,
    output reg       sampled
);
    localparam S_SYNC = 2'd0, S_TSEG1 = 2'd1, S_TSEG2 = 2'd2;

    reg [5:0] presc;
    reg [1:0] seg;
    reg [4:0] seg_cnt;
    reg [2:0] ext;
    reg [3:0] ph2_len;
    reg       rx_prev, synced;

    wire       tq_tick = (presc == brp);
    wire [2:0] sjw     = {1'b0, sjw_m1} + 3'd1;
    wire [3:0] ph2     = {1'b0, ph2_m1} + 4'd1;
    wire [4:0] tseg1   = {2'b00, prop_m1} + {2'b00, ph1_m1} + 5'd2 + {2'b00, ext};
    wire       fall    = rx_prev && !rx_s && sampled;
    wire       rsync   = fall && resync_en && !synced;
```

```
    wire [4:0] lag     = seg_cnt + 5'd1;
    wire [4:0] lead    = {1'b0, ph2_len} - seg_cnt;
    wire       restart = (fall && hard_sync_en) ||
                         (rsync && seg == S_TSEG2 && lead <= {2'b00, sjw});

    always @(posedge clk) begin
        tx_point     <= 1'b0;
        sample_point <= 1'b0;
        rx_prev      <= rx_s;
        if (reset || !enable) begin
            presc <= 6'd0;  seg <= S_SYNC;  seg_cnt <= 5'd0;  ext <= 3'd0;
            ph2_len <= ph2;  synced <= 1'b0;  sampled <= 1'b1;  rx_prev <= 1'b1;
        end
        else if (restart) begin
            presc <= 6'd0;  seg <= S_SYNC;  seg_cnt <= 5'd0;  ext <= 3'd0;
            ph2_len <= ph2;  synced <= 1'b1;  tx_point <= 1'b1;
        end
        else begin
            if (rsync) begin
                synced <= 1'b1;
                if (seg == S_TSEG1)      ext     <= (lag > {2'b00, sjw}) ? sjw : lag[2:0];
                else if (seg == S_TSEG2) ph2_len <= ph2_len - {1'b0, sjw};
            end
            if (tq_tick) begin
                presc <= 6'd0;
                case (seg)
                    S_SYNC: begin seg <= S_TSEG1; seg_cnt <= 5'd0; end
                    S_TSEG1:
                        if (seg_cnt >= tseg1 - 5'd1) begin
                            seg <= S_TSEG2;  seg_cnt <= 5'd0;
                            sample_point <= 1'b1;
                            sampled      <= rx_s;
                        end
                        else seg_cnt <= seg_cnt + 5'd1;
                    default:
                        if (seg_cnt >= {1'b0, ph2_len} - 5'd1) begin
                            seg <= S_SYNC;  seg_cnt <= 5'd0;  ext <= 3'd0;
                            ph2_len <= ph2;  synced <= 1'b0;  tx_point <= 1'b1;
                        end
                        else seg_cnt <= seg_cnt + 5'd1;
                endcase
            end
            else presc <= presc + 6'd1;
        end
    end
endmodule

module CAN_CRC15 (
    input             clk,
    input             reset,
    input             clear,
    input             enable,
    input             bit_in,
    output reg [14:0] crc
);
    wire crc_nxt = bit_in ^ crc[14];

    always @(posedge clk) begin
        if (reset || clear)
            crc <= 15'd0;
        else if (enable)
            crc <= {crc[13:0], 1'b0} ^ (crc_nxt ? 15'h4599 : 15'h0000);
    end
endmodule
```

V. Testbench Code

```
`timescale 1ns/10ps

module CAN_CONTROLLER_TB;

    localparam BIT_CLKS = 40;
`ifdef GATE
    localparam B_MBOX = 2, B_FILT = 1, B_DEPTH = 1;
`else
    localparam B_MBOX = 3, B_FILT = 2, B_DEPTH = 3;
`endif
    localparam NA = 0, NB = 1, NC = 2;

    reg clk_a = 1'b0, clk_b = 1'b0, reset = 1'b1;
    always #5    clk_a = ~clk_a;
    always #5.05 clk_b = ~clk_b;

    reg  [7:0] a_addr = 0, a_wdata = 0, b_addr = 0, b_wdata = 0, c_addr = 0, c_wdata = 0;
    reg        a_we = 0, b_we = 0, c_we = 0;
    wire [7:0] a_rdata, b_rdata, c_rdata;
    wire       a_irq, b_irq, c_irq, a_tx, b_tx, c_tx;
    reg        jam_n = 1'b1;
    wire       bus = a_tx & b_tx & c_tx & jam_n;

    CAN_CONTROLLER nodeA (.clk(clk_a), .reset(reset), .addr(a_addr), .wdata(a_wdata),
                          .we(a_we), .rdata(a_rdata), .irq(a_irq), .rx(bus), .tx(a_tx));
`ifdef GATE
    CAN_CONTROLLER nodeB (.clk(clk_b), .reset(reset), .addr(b_addr), .wdata(b_wdata),
                          .we(b_we), .rdata(b_rdata), .irq(b_irq), .rx(bus), .tx(b_tx));
`else
    CAN_CONTROLLER #(.NUM_TX_MBOX(B_MBOX), .NUM_FILTERS(B_FILT), .RX_DEPTH(B_DEPTH))
                   nodeB (.clk(clk_b), .reset(reset), .addr(b_addr), .wdata(b_wdata),
                          .we(b_we), .rdata(b_rdata), .irq(b_irq), .rx(bus), .tx(b_tx));
`endif
    CAN_CONTROLLER nodeC (.clk(clk_a), .reset(reset), .addr(c_addr), .wdata(c_wdata),
                          .we(c_we), .rdata(c_rdata), .irq(c_irq), .rx(bus), .tx(c_tx));

    integer errors = 0;
    reg [7:0] d, d2;

    task wr(input integer n, input [7:0] ad, input [7:0] dat);
        begin
            if (n == NA) begin
                @(negedge clk_a); a_addr = ad; a_wdata = dat; a_we = 1;
                @(negedge clk_a); a_we = 0;
            end
            else if (n == NB) begin
                @(negedge clk_b); b_addr = ad; b_wdata = dat; b_we = 1;
                @(negedge clk_b); b_we = 0;
            end
            else begin
                @(negedge clk_a); c_addr = ad; c_wdata = dat; c_we = 1;
                @(negedge clk_a); c_we = 0;
            end
        end
    endtask

    task rd(input integer n, input [7:0] ad, output [7:0] dat);
        begin
            if (n == NA)      begin @(negedge clk_a); a_addr = ad; #1 dat = a_rdata; end
            else if (n == NB) begin @(negedge clk_b); b_addr = ad; #1 dat = b_rdata; end
            else              begin @(negedge clk_a); c_addr = ad; #1 dat = c_rdata; end
        end
    endtask

    task check(input cond, input [8*80-1:0] msg);
        begin
            if (cond) $display("  ok    %0s", msg);
            else begin $display("  FAIL  %0s", msg); errors = errors + 1; end
        end
    endtask

    function [28:0] sid(input [10:0] id);
        sid = {id, 18'd0};
    endfunction
```

```
    task load_mb(input integer n, input integer mb, input [28:0] id, input ide, input rtr,
                 input [3:0] dlc, input [63:0] data);
        reg [7:0] base; integer j;
        begin
            base = 8'h10 + 8'h10 * mb;
            wr(n, base + 1, {3'b000, id[28:24]});
            wr(n, base + 2, id[23:16]);
            wr(n, base + 3, id[15:8]);
            wr(n, base + 4, id[7:0]);
            wr(n, base + 5, {ide, rtr, 2'b00, dlc});
            for (j = 0; j < 8; j = j + 1) wr(n, base + 6 + j, data[63-8*j -: 8]);
        end
    endtask

    task request(input integer n, input integer mb);
        wr(n, 8'h10 + 8'h10 * mb, 8'h01);
    endtask

    task wait_tx(input integer n, input integer mb, input integer max_bits, output ok);
        integer t;
        begin
            ok = 0;
            for (t = 0; t < max_bits * 4 && !ok; t = t + 1) begin
                repeat (BIT_CLKS / 4) @(posedge clk_a);
                rd(n, 8'h10 + 8'h10 * mb, d);
                if (!d[0]) ok = 1;
            end
        end
    endtask

    task wait_rx(input integer n, input integer max_bits, output ok);
        integer t;
        begin
            ok = 0;
            for (t = 0; t < max_bits * 4 && !ok; t = t + 1) begin
                repeat (BIT_CLKS / 4) @(posedge clk_a);
                rd(n, 8'h01, d);
                if (d[5]) ok = 1;
            end
        end
    endtask

    task read_rx(input integer n, output [28:0] id, output ide, output rtr,
                 output [3:0] dlc, output [63:0] data, output [3:0] hit);
        integer j;
        begin
            rd(n, 8'h51, d); id[28:24] = d[4:0];
            rd(n, 8'h52, d); id[23:16] = d;
            rd(n, 8'h53, d); id[15:8]  = d;
            rd(n, 8'h54, d); id[7:0]   = d;
            rd(n, 8'h55, d); ide = d[7]; rtr = d[6]; dlc = d[3:0];
            for (j = 0; j < 8; j = j + 1) begin rd(n, 8'h56 + j, d); data[63-8*j -: 8] = d; end
            rd(n, 8'h5E, d); hit = d[3:0];
        end
    endtask

    task expect_rx(input integer n, input [28:0] id, input ide, input rtr, input [3:0] dlc,
                   input [63:0] data, input [8*80-1:0] msg);
        reg ok; reg [28:0] g_id; reg g_ide, g_rtr; reg [3:0] g_dlc, g_hit; reg [63:0] g_data;
        begin
            wait_rx(n, 400, ok);
            if (!ok) check(0, msg);
            else begin
                read_rx(n, g_id, g_ide, g_rtr, g_dlc, g_data, g_hit);
                wr(n, 8'h50, 8'h01);
                if (g_id == id && g_ide == ide && g_rtr == rtr && g_dlc == dlc && g_data == data)
                    check(1, msg);
                else begin
                    check(0, msg);
                    $display("        got id=%h ide=%b rtr=%b dlc=%0d data=%h", g_id, g_ide, g_rtr, g_dlc, g_data);
                    $display("        exp id=%h ide=%b rtr=%b dlc=%0d data=%h", id, ide, rtr, dlc, data);
                end
            end
        end
    endtask

    reg     exp_bits [0:255];
    integer exp_len;
```

```
    task build_frame(input [28:0] id, input ide, input rtr, input [3:0] dlc,
                     input [63:0] data, input ack);
        reg raw [0:255]; integer rl, j, run; reg [14:0] crc; reg nxt, last;
        begin
            rl = 0;
            raw[rl] = 1'b0; rl = rl + 1;
            for (j = 28; j >= 18; j = j - 1) begin raw[rl] = id[j]; rl = rl + 1; end
            if (ide) begin
                raw[rl] = 1'b1; rl = rl + 1;
                raw[rl] = 1'b1; rl = rl + 1;
                for (j = 17; j >= 0; j = j - 1) begin raw[rl] = id[j]; rl = rl + 1; end
                raw[rl] = rtr;  rl = rl + 1;
                raw[rl] = 1'b0; rl = rl + 1;
            end
            else begin
                raw[rl] = rtr;  rl = rl + 1;
                raw[rl] = 1'b0; rl = rl + 1;
            end
            raw[rl] = 1'b0; rl = rl + 1;
            for (j = 3; j >= 0; j = j - 1) begin raw[rl] = dlc[j]; rl = rl + 1; end
            if (!rtr)
                for (j = 0; j < ((dlc > 8) ? 64 : dlc * 8); j = j + 1) begin
                    raw[rl] = data[63 - j]; rl = rl + 1;
                end
            crc = 15'd0;
            for (j = 0; j < rl; j = j + 1) begin
                nxt = raw[j] ^ crc[14];
                crc = {crc[13:0], 1'b0} ^ (nxt ? 15'h4599 : 15'h0000);
            end
            for (j = 14; j >= 0; j = j - 1) begin raw[rl] = crc[j]; rl = rl + 1; end
            exp_len = 0; run = 0; last = 1'b0;
            for (j = 0; j < rl; j = j + 1) begin
                exp_bits[exp_len] = raw[j]; exp_len = exp_len + 1;
                if (j == 0 || raw[j] != last) begin last = raw[j]; run = 1; end
                else run = run + 1;
                if (run == 5) begin
                    exp_bits[exp_len] = ~last; exp_len = exp_len + 1;
                    last = ~last; run = 1;
                end
            end
            exp_bits[exp_len] = 1'b1;  exp_len = exp_len + 1;
            exp_bits[exp_len] = !ack;  exp_len = exp_len + 1;
            for (j = 0; j < 8; j = j + 1) begin
                exp_bits[exp_len] = 1'b1; exp_len = exp_len + 1;
            end
        end
    endtask

    reg     mon_arm = 1'b0, mon_done = 1'b0;
    reg     cap [0:255];
    integer mi;
    always begin
        wait (mon_arm);
        @(negedge bus);
        mon_arm = 1'b0;
        repeat (BIT_CLKS / 2) @(posedge clk_a);
        for (mi = 0; mi < exp_len; mi = mi + 1) begin
            cap[mi] = bus;
            if (mi < exp_len - 1) repeat (BIT_CLKS) @(posedge clk_a);
        end
        mon_done = 1'b1;
    end

    task check_stream(input [8*80-1:0] msg);
        integer j, bad;
        begin
            wait (mon_done);
            mon_done = 1'b0;
            bad = -1;
            for (j = exp_len - 1; j >= 0; j = j - 1) if (cap[j] !== exp_bits[j]) bad = j;
            check(bad == -1, msg);
            if (bad != -1) $display("        first mismatch at bit %0d of %0d", bad, exp_len);
        end
    endtask

    reg     jam_en = 1'b0;
    integer jam_idx, idle_cnt;
```

```
    always begin
        wait (jam_en);
        idle_cnt = 0;
        while (idle_cnt < 10 * BIT_CLKS) begin
            @(posedge clk_a);
            if (bus) idle_cnt = idle_cnt + 1; else idle_cnt = 0;
        end
        @(negedge bus);
        if (jam_en) begin
            repeat (jam_idx * BIT_CLKS) @(posedge clk_a);
            jam_n = 1'b0;
            repeat (BIT_CLKS) @(posedge clk_a);
            jam_n = 1'b1;
        end
    end

    task enable_node(input integer n);
        integer t;
        begin
            wr(n, 8'h00, 8'h01);
            d = 0;
            for (t = 0; t < 100 && !d[7]; t = t + 1) begin
                repeat (BIT_CLKS / 2) @(posedge clk_a);
                rd(n, 8'h01, d);
            end
        end
    endtask

    reg ok, ok2;
    reg [28:0] g_id; reg g_ide, g_rtr; reg [3:0] g_dlc, g_hit; reg [63:0] g_data;
    integer i, t, cnt;

    initial begin
      `ifdef VCD_FILE
        $dumpfile(`VCD_FILE);
      `else
        $dumpfile("dump.vcd");
      `endif
        $dumpvars(0, CAN_CONTROLLER_TB);

        repeat (5) @(posedge clk_b);
        reset = 1'b0;

        $display("T1  register defaults and read/write");
        rd(NA, 8'h04, d); check(d == 8'h43, "BTR0 reset = 0x43 (SJW 2, BRP 3)");
        rd(NA, 8'h05, d); check(d == 8'h11, "BTR1 reset = 0x11 (PROP 2, PH1 3)");
        rd(NA, 8'h06, d); check(d == 8'h03, "BTR2 reset = 0x03 (PH2 4)");
        wr(NA, 8'h02, 8'hA5); rd(NA, 8'h02, d); check(d == 8'hA5, "INT_EN write/read");
        wr(NA, 8'h02, 8'hFF);
        load_mb(NA, 0, 29'h12345678, 1'b1, 1'b0, 4'd5, 64'h0102030405060708);
        rd(NA, 8'h12, d); ok = (d == 8'h34);
        rd(NA, 8'h15, d); ok = ok && (d == 8'h85);
        rd(NA, 8'h1A, d); check(ok && d == 8'h05, "mailbox ID/control/data readback");

        $display("T2  loopback self-test (node A alone)");
        wr(NA, 8'h00, 8'h03);
        repeat (15 * BIT_CLKS) @(posedge clk_a);
        load_mb(NA, 0, sid(11'h3C5), 1'b0, 1'b0, 4'd3, 64'hDEADBE0000000000);
        request(NA, 0);
        wait_tx(NA, 0, 300, ok); check(ok, "loopback frame sent without ACK error");
        check(bus === 1'b1, "tx pin stays recessive in loopback");
        expect_rx(NA, sid(11'h3C5), 1'b0, 1'b0, 4'd3, 64'hDEADBE0000000000, "loopback frame received");
        rd(NA, 8'h07, d); check(d == 0, "TEC = 0 after loopback");
        wr(NA, 8'h00, 8'h00);
        wr(NA, 8'h03, 8'hFF);

        $display("T3  A -> B standard data frame, exact bitstream");
        enable_node(NA); enable_node(NB); enable_node(NC);
        rd(NB, 8'h01, d); check(d[7], "all nodes integrated (bus idle)");
        wr(NB, 8'h02, 8'hFF);
        load_mb(NA, 0, sid(11'h123), 1'b0, 1'b0, 4'd2, 64'h55EE000000000000);
        build_frame(sid(11'h123), 1'b0, 1'b0, 4'd2, 64'h55EE000000000000, 1'b1);
        mon_arm = 1'b1;
        request(NA, 0);
        wait_tx(NA, 0, 300, ok); check(ok, "TXREQ cleared after transmission");
        check_stream("bus bits match reference (CRC, stuffing, ACK)");
        rd(NA, 8'h03, d); check(d[0] && a_irq, "A: TX0 done flag and irq");
        rd(NB, 8'h03, d); check(d[4] && b_irq, "B: RX flag and irq");
```

```
        expect_rx(NB, sid(11'h123), 1'b0, 1'b0, 4'd2, 64'h55EE000000000000, "B received the frame");
        expect_rx(NC, sid(11'h123), 1'b0, 1'b0, 4'd2, 64'h55EE000000000000, "C received the frame");
        wr(NA, 8'h03, 8'hFF); wr(NB, 8'h03, 8'hFF); wr(NC, 8'h03, 8'hFF);
        check(!a_irq && !b_irq, "INT_FLAG write-1-to-clear drops irq");

        $display("T4  A -> B extended 8-byte frame, exact bitstream");
        load_mb(NA, 0, 29'h12345678, 1'b1, 1'b0, 4'd8, 64'h0000FFFF00FF55AA);
        build_frame(29'h12345678, 1'b1, 1'b0, 4'd8, 64'h0000FFFF00FF55AA, 1'b1);
        mon_arm = 1'b1;
        request(NA, 0);
        wait_tx(NA, 0, 300, ok); check(ok, "extended frame sent");
        check_stream("bus bits match reference");
        expect_rx(NB, 29'h12345678, 1'b1, 1'b0, 4'd8, 64'h0000FFFF00FF55AA, "B received the extended frame");
        expect_rx(NC, 29'h12345678, 1'b1, 1'b0, 4'd8, 64'h0000FFFF00FF55AA, "C received the extended frame");

        $display("T5  remote frames and DLC 0");
        load_mb(NA, 0, sid(11'h7EF), 1'b0, 1'b1, 4'd4, 64'hFFFFFFFFFFFFFFFF);
        build_frame(sid(11'h7EF), 1'b0, 1'b1, 4'd4, 64'h0, 1'b1);
        mon_arm = 1'b1;
        request(NA, 0);
        wait_tx(NA, 0, 300, ok);
        check_stream("standard remote frame: DLC 4, no data field");
        expect_rx(NB, sid(11'h7EF), 1'b0, 1'b1, 4'd4, 64'h0, "B received remote frame");
        expect_rx(NC, sid(11'h7EF), 1'b0, 1'b1, 4'd4, 64'h0, "C received remote frame");
        load_mb(NA, 0, 29'h1FFFFFFF, 1'b1, 1'b0, 4'd0, 64'h0);
        build_frame(29'h1FFFFFFF, 1'b1, 1'b0, 4'd0, 64'h0, 1'b1);
        mon_arm = 1'b1;
        request(NA, 0);
        wait_tx(NA, 0, 300, ok);
        check_stream("extended data frame with DLC 0");
        expect_rx(NB, 29'h1FFFFFFF, 1'b1, 1'b0, 4'd0, 64'h0, "B received DLC 0 frame");
        expect_rx(NC, 29'h1FFFFFFF, 1'b1, 1'b0, 4'd0, 64'h0, "C received DLC 0 frame");

        $display("T6  B -> A (B's clock is 1%% slow)");
        load_mb(NB, 0, sid(11'h0F0), 1'b0, 1'b0, 4'd8, 64'h00000000FFFFFFFF);
        request(NB, 0);
        wait_tx(NB, 0, 300, ok); check(ok, "B's frame sent");
        expect_rx(NA, sid(11'h0F0), 1'b0, 1'b0, 4'd8, 64'h00000000FFFFFFFF, "A received B's frame");
        expect_rx(NC, sid(11'h0F0), 1'b0, 1'b0, 4'd8, 64'h00000000FFFFFFFF, "C received B's frame");
        load_mb(NB, 0, 29'h0ABCDEF1, 1'b1, 1'b0, 4'd8, 64'h8000000000000001);
        request(NB, 0);
        wait_tx(NB, 0, 300, ok);
        expect_rx(NA, 29'h0ABCDEF1, 1'b1, 1'b0, 4'd8, 64'h8000000000000001, "A received B's extended frame");
        expect_rx(NC, 29'h0ABCDEF1, 1'b1, 1'b0, 4'd8, 64'h8000000000000001, "C received B's extended frame");

        $display("T7  arbitration: A and B start in the same bit");

        wr(NA, 8'h81, 8'h00); wr(NA, 8'h82, 8'h00); wr(NA, 8'h83, 8'h00); wr(NA, 8'h84, 8'h00);
        wr(NA, 8'h85, 8'h10); wr(NA, 8'h86, 8'h00); wr(NA, 8'h87, 8'h00); wr(NA, 8'h88, 8'h00);
        wr(NA, 8'h80, 8'h01);

        load_mb(NA, 0, sid(11'h155), 1'b0, 1'b0, 4'd1, 64'hAA00000000000000);
        load_mb(NB, 0, sid(11'h154), 1'b0, 1'b0, 4'd1, 64'hBB00000000000000);
        load_mb(NC, 0, sid(11'h7FF), 1'b0, 1'b0, 4'd8, 64'h1111111111111111);
        request(NC, 0);
        rd(NC, 8'h01, d);
        while (!d[0]) rd(NC, 8'h01, d);
        request(NA, 0); request(NB, 0);
        wait_tx(NB, 0, 400, ok);
        wr(NB, 8'h50, 8'h01);
        rd(NA, 8'h10, d); check(ok && d[0] && d[1], "B (0x154) won; A still pending, lost flag set");
        wait_tx(NA, 0, 400, ok); check(ok, "A retransmitted after losing");
        expect_rx(NA, sid(11'h154), 1'b0, 1'b0, 4'd1, 64'hBB00000000000000, "A: received winner B");
        expect_rx(NB, sid(11'h155), 1'b0, 1'b0, 4'd1, 64'hAA00000000000000, "B: received A after");
        for (i = 0; i < 3; i = i + 1) wr(NC, 8'h50, 8'h01);

        load_mb(NA, 0, 29'h0AA80001, 1'b1, 1'b0, 4'd1, 64'hEE00000000000000);
        load_mb(NB, 0, sid(11'h2AA),  1'b0, 1'b0, 4'd1, 64'h5500000000000000);
        request(NC, 0);
        rd(NC, 8'h01, d);
        while (!d[0]) rd(NC, 8'h01, d);
        request(NA, 0); request(NB, 0);
        wait_tx(NB, 0, 400, ok);
        wr(NB, 8'h50, 8'h01);
        rd(NA, 8'h10, d); check(ok && d[0] && d[1], "standard 0x2AA beat extended with base 0x2AA");
        wait_tx(NA, 0, 400, ok); check(ok, "extended frame followed");
        expect_rx(NA, sid(11'h2AA), 1'b0, 1'b0, 4'd1, 64'h5500000000000000, "A: received standard winner");
        expect_rx(NB, 29'h0AA80001, 1'b1, 1'b0, 4'd1, 64'hEE00000000000000, "B: received extended frame");
```

```
        for (i = 0; i < 3; i = i + 1) wr(NC, 8'h50, 8'h01);
        rd(NA, 8'h07, d); rd(NB, 8'h07, d2);
        check(d == 0 && d2 == 0, "no errors: TEC = 0 on A and B");

        if (B_MBOX >= 3) begin
            $display("T8  mailbox priority on B (lowest ID first)");
            load_mb(NB, 0, sid(11'h300), 1'b0, 1'b0, 4'd1, 64'h3000000000000000);
            load_mb(NB, 1, sid(11'h100), 1'b0, 1'b0, 4'd1, 64'h1000000000000000);
            load_mb(NB, 2, sid(11'h200), 1'b0, 1'b0, 4'd1, 64'h2000000000000000);
            request(NC, 0);
            rd(NC, 8'h01, d);
            while (!d[0]) rd(NC, 8'h01, d);
            request(NB, 0); request(NB, 1); request(NB, 2);
            expect_rx(NA, sid(11'h100), 1'b0, 1'b0, 4'd1, 64'h1000000000000000, "first 0x100 (mailbox 1)");
            expect_rx(NA, sid(11'h200), 1'b0, 1'b0, 4'd1, 64'h2000000000000000, "then 0x200 (mailbox 2)");
            expect_rx(NA, sid(11'h300), 1'b0, 1'b0, 4'd1, 64'h3000000000000000, "then 0x300 (mailbox 0)");
            rd(NB, 8'h03, d); check(d[2:0] == 3'b111, "B: TX done flags for all three mailboxes");
            wr(NB, 8'h03, 8'hFF);
            wr(NB, 8'h50, 8'h01);
            for (i = 0; i < 4; i = i + 1) wr(NC, 8'h50, 8'h01);
        end

        wr(NA, 8'h80, 8'h00);
        $display("T9  acceptance filters on B");

        wr(NB, 8'h81, 8'h04); wr(NB, 8'h82, 8'h80); wr(NB, 8'h83, 8'h00); wr(NB, 8'h84, 8'h00);
        wr(NB, 8'h85, 8'h1F); wr(NB, 8'h86, 8'hC0); wr(NB, 8'h87, 8'h00); wr(NB, 8'h88, 8'h00);
        wr(NB, 8'h80, 8'h01);
        if (B_FILT >= 2) begin
            wr(NB, 8'h91, 8'h1A); wr(NB, 8'h92, 8'hBC); wr(NB, 8'h93, 8'hDE); wr(NB, 8'h94, 8'h00);
            wr(NB, 8'h95, 8'h1F); wr(NB, 8'h96, 8'hFF); wr(NB, 8'h97, 8'hFF); wr(NB, 8'h98, 8'h00);
            wr(NB, 8'h90, 8'h03);
        end
        rd(NB, 8'h82, d); check(d == 8'h80, "filter register readback");
        load_mb(NA, 0, sid(11'h135), 1'b0, 1'b0, 4'd1, 64'h0100000000000000);
        request(NA, 0); wait_tx(NA, 0, 300, ok);
        check(ok, "rejected frame is still acknowledged by B");
        load_mb(NA, 0, 29'h04840000, 1'b1, 1'b0, 4'd1, 64'h0200000000000000);
        request(NA, 0); wait_tx(NA, 0, 300, ok);
        load_mb(NA, 0, sid(11'h125), 1'b0, 1'b0, 4'd1, 64'h0300000000000000);
        request(NA, 0); wait_tx(NA, 0, 300, ok);
        load_mb(NA, 0, 29'h1ABCDE55, 1'b1, 1'b0, 4'd1, 64'h0400000000000000);
        request(NA, 0); wait_tx(NA, 0, 300, ok);
        rd(NB, 8'h50, d);
        check(d[4:2] == ((B_FILT >= 2) ? 2 : 1), "B stored only the matching frames");
        read_rx(NB, g_id, g_ide, g_rtr, g_dlc, g_data, g_hit); wr(NB, 8'h50, 8'h01);
        check(g_id == sid(11'h125) && g_hit == 4'b1000, "0x125 stored, FILHIT = filter 0");
        if (B_FILT >= 2) begin
            read_rx(NB, g_id, g_ide, g_rtr, g_dlc, g_data, g_hit); wr(NB, 8'h50, 8'h01);
            check(g_id == 29'h1ABCDE55 && g_hit == 4'b1001, "0x1ABCDE55 stored, FILHIT = filter 1");
        end
        wr(NB, 8'h80, 8'h00); wr(NB, 8'h90, 8'h00);
        for (i = 0; i < 4; i = i + 1) wr(NC, 8'h50, 8'h01);
        wr(NA, 8'h03, 8'hFF); wr(NB, 8'h03, 8'hFF);

        $display("T10 receive FIFO overflow on A (depth 1)");
        load_mb(NB, 0, sid(11'h011), 1'b0, 1'b0, 4'd1, 64'h1100000000000000);
        request(NB, 0); wait_tx(NB, 0, 300, ok);
        load_mb(NB, 0, sid(11'h022), 1'b0, 1'b0, 4'd1, 64'h2200000000000000);
        request(NB, 0); wait_tx(NB, 0, 300, ok);
        rd(NA, 8'h50, d); check(d[1] && d[4:2] == 1, "overflow flagged, one frame kept");
        rd(NA, 8'h03, d); check(d[5], "RX overflow interrupt flag");
        expect_rx(NA, sid(11'h011), 1'b0, 1'b0, 4'd1, 64'h1100000000000000, "oldest frame kept");
        wr(NA, 8'h50, 8'h02); rd(NA, 8'h50, d); check(d == 0, "overflow cleared, FIFO empty");
        for (i = 0; i < 2; i = i + 1) wr(NC, 8'h50, 8'h01);
        wr(NA, 8'h03, 8'hFF);

        $display("T11 no receiver: ACK errors, error passive");
        wr(NB, 8'h00, 8'h00); wr(NC, 8'h00, 8'h00);
        load_mb(NA, 0, sid(11'h444), 1'b0, 1'b0, 4'd1, 64'h4400000000000000);
        request(NA, 0);
        ok = 0;
        for (t = 0; t < 4000 && !ok; t = t + 1) begin
            repeat (BIT_CLKS / 2) @(posedge clk_a);
            rd(NA, 8'h01, d); ok = d[3];
        end
        rd(NA, 8'h07, d); check(ok && d == 128, "error passive after 16 ACK errors (TEC = 128)");
        rd(NA, 8'h09, d); check(d == 5, "ERR_CODE = ACK error");
```

```
        rd(NA, 8'h10, d); check(d == 8'h05, "mailbox still pending with error flag");
        repeat (300 * BIT_CLKS) @(posedge clk_a);
        rd(NA, 8'h07, d); check(d == 128, "passive ACK errors do not raise TEC");
        wr(NA, 8'h10, 8'h00);
        repeat (150 * BIT_CLKS) @(posedge clk_a);
        rd(NA, 8'h01, d); check(d[7] && !d[0], "abort: bus idle, A no longer transmitting");

        $display("T12 bit errors -> bus-off -> recovery");
        wr(NA, 8'h00, 8'h00);
        enable_node(NA); enable_node(NB); enable_node(NC);
        wr(NA, 8'h03, 8'hFF);
        load_mb(NA, 0, sid(11'h0A0), 1'b0, 1'b0, 4'd2, 64'hFFFF000000000000);
        build_frame(sid(11'h0A0), 1'b0, 1'b0, 4'd2, 64'hFFFF000000000000, 1'b1);
        jam_idx = 21;
        while (exp_bits[jam_idx] !== 1'b1) jam_idx = jam_idx + 1;
        jam_en = 1'b1;
        repeat (12 * BIT_CLKS) @(posedge clk_a);
        request(NA, 0);
        ok = 0;
        for (t = 0; t < 8000 && !ok; t = t + 1) begin
            repeat (BIT_CLKS / 2) @(posedge clk_a);
            rd(NA, 8'h01, d); ok = d[4];
        end
        jam_en = 1'b0;
        rd(NA, 8'h07, d); check(ok && d == 8'hFF, "bus-off after 32 bit errors (TEC > 255)");
        rd(NA, 8'h09, d); check(d == 1, "ERR_CODE = bit error");
        rd(NA, 8'h03, d); check(d[7] && d[6], "bus-off and error interrupt flags");
        rd(NB, 8'h08, d); check(d > 0 && d < 128, "B's REC counted the errors, B still error active");
        ok2 = 0;
        for (t = 0; t < 8000 && !ok2; t = t + 1) begin
            repeat (BIT_CLKS / 2) @(posedge clk_a);
            rd(NA, 8'h01, d); ok2 = !d[4];
        end
        rd(NA, 8'h07, d);
        check(ok2 && d == 0, "recovered after 128 x 11 recessive bits, TEC = 0");
        wait_tx(NA, 0, 300, ok); check(ok, "pending frame sent after recovery");
        expect_rx(NB, sid(11'h0A0), 1'b0, 1'b0, 4'd2, 64'hFFFF000000000000, "B received it");

        if (errors == 0) $display("ALL TESTS PASSED");
        else             $display("%0d CHECK(S) FAILED", errors);
        $finish;
    end

    initial begin
        #20_000_000;
        $display("TIMEOUT");
        $finish;
    end

endmodule
```